講義 26：新式語法總覽——字串模板 / New Open SQL / Functional Operator（授課順序：進階選修，全課程最後一講，接在講義 23 之後）

> 對應練習：ex26 | 答案程式：ZR_TR26_MODERN_SYNTAX

本講重點

- 行內宣告（Inline Declaration）：`DATA(...)` / `FIELD-SYMBOL(...)`——後面每一種新式寫法都會用到
- 字串模板 `|...|`：取代 `CONCATENATE`，並附格式化選項（`WIDTH` / `ALIGN` / `PAD` / `CASE`）
- New Open SQL：宿主變數（Host Variable）與 `@` 跳脫規則、`@DATA(...)` 行內宣告、`SELECT` 清單裡的 `CASE` 運算式
- Functional Operator 五件套：`COND` / `SWITCH` / `VALUE` / `REDUCE` / `FILTER`
- 定位：這些都是 7.40 之後（S/4HANA 常態寫法）的語法。各語法點已經在教到它的講義裡，以「新式寫法對照」區塊跟傳統寫法並列說明（位置見第 8 節速查表）；這一講把它們集中起來當**總整理與速查**，並補上 `SELECT` 裡的 `CASE` 運算式、字串模板格式選項、`FILTER` 次要鍵等進階內容——**兩種寫法都要看得懂**，新專案/S4 常見新式寫法，維護舊程式仍要認得傳統寫法。

0. 先備觀念：行內宣告（Inline Declaration）

前面 25 講的變數、work area、field-symbol 都是「**先宣告、再使用**」。7.40 起，ABAP 允許在**第一次使用的地方**直接宣告，這叫**行內宣告（Inline Declaration）**——本講後面的每一種新式寫法幾乎都會用到，所以先把它講清楚。

形式	用在哪裡	型別怎麼來
`DATA(名稱)`	賦值左邊、`LOOP AT ... INTO`、`READ TABLE ... INTO`、方法呼叫的接收端	由右邊的值 / 來源自動推導
`FIELD-SYMBOL(<名稱>)`	`LOOP AT ... ASSIGNING`、`READ TABLE ... ASSIGNING`、`ASSIGN ... TO`	內表的列型別（`ASSIGN COMPONENT` 為 `any`）
`@DATA(名稱)`	`SELECT ... INTO`（第 2 節）	由 `SELECT` 清單推導

```
* 傳統寫法：先宣告，再使用
DATA gv_len TYPE i.
gv_len = strlen( 'ABAP' ).

DATA gs_student TYPE ty_student.
LOOP AT gt_students INTO gs_student.
  ...
ENDLOOP.

FIELD-SYMBOLS <ls_student> TYPE ty_student.
LOOP AT gt_students ASSIGNING <ls_student>.
```

```
  ...
ENDLOOP.

* 行內宣告：宣告與使用合在同一行
DATA(gv_len) = strlen( 'ABAP' ).

LOOP AT gt_students INTO DATA(gs_student).
  ...
ENDLOOP.

LOOP AT gt_students ASSIGNING FIELD-SYMBOL(<ls_student>).
  ...
ENDLOOP.
```

四個要注意的地方（都已在 SAP 系統實測）：

1. **型別由右邊決定，字面值（Literal）容易誤判**：`DATA(lv_n) = 0.` 是 `i`，之後 `lv_n = 7 / 2.` 得 `4`（整數四捨五入）；`DATA(lv_s) = 'abc'.` 推導出的是 `c LENGTH 3`，之後 `lv_s = 'abcdefgh'.` 只剩 `abc`（被截斷，講義 1 提過的固定長度文字）；要 `string` 就得用反引號 `` DATA(lv_s) = abc. ``。
2. **作用範圍是整個程式單元**（程式主體 / FORM / 方法），不是只有 `LOOP` 裡面：兩個 `LOOP` 都寫 `INTO DATA(gv_x)`，第二個會編譯錯誤 `"GV_X" was already declared`——換不同名稱，或第二個 `LOOP` 直接 `INTO gv_x`。
3. **不是每個位置都能行內宣告**：`CALL FUNCTION ... IMPORTING x = DATA(y)` 不合法（Function Module 的參數一律要事先宣告好的變數），只有方法呼叫等少數位置支援。
4. **只是少寫一行宣告，效果與傳統寫法完全相同**，不影響執行結果與效能；讀新專案 / S/4HANA 程式碼時兩種都要看得懂。各講義的「新式寫法對照」都用到行內宣告，第一次介紹在講義 2 §1。

1. 字串模板（String Template）

```
DATA: gv_last  TYPE c LENGTH 10 VALUE '王',
      gv_first TYPE c LENGTH 10 VALUE '小明',
      gv_score TYPE i VALUE 85,
      gv_code  TYPE c LENGTH 3 VALUE 'lh'.

WRITE: / |全名：{ gv_last }{ gv_first }|.
* 等同 CONCATENATE gv_last gv_first INTO ...，但可以直接寫在 WRITE/字串運算式裡，不用先宣告暫存變數

WRITE: / |成績（靠右對齊，補零至 5 碼）：{ gv_score WIDTH = 5 ALIGN = RIGHT PAD = '0' }|.
* 00085

WRITE: / |代碼轉大寫：{ gv_code CASE = UPPER }|.
* LH
```

- 基本語法：`|文字{ 運算式 }文字|`，`{ }` 內可以放任何單一運算式（變數、四則運算、`RETURNING` 的方法呼叫），**不能**是多參數的 `EXPORTING`/`IMPORTING` 方法呼叫。
- 常用格式化選項：

選項	作用
`WIDTH = n`	輸出寬度
`ALIGN = LEFT / RIGHT / CENTER`	對齊方式
`PAD = 'x'`	補齊字元（預設補空白）
`CASE = UPPER / LOWER`	轉大小寫（不加括號；`(UPPER)` 是動態參照語法，會當成變數名報 `Field "UPPER" is unknown`）
`DECIMALS = n`	數值小數位數

- 對照講義 18：`CONCATENATE` 要先想好暫存變數、拼接多段要嵌套或用 `SEPARATED BY`；字串模板可以把文字與變數交錯寫在同一行，可讀性通常更好，尤其是組訊息文字（`MESSAGE`、ALV 欄位標題）。

2. New Open SQL：行內宣告與 SELECT 內的 CASE 運算式

2.1 宿主變數（Host Variable）：是什麼、為什麼需要 `@`

一句 `SELECT` 裡同時會出現兩種名稱：**資料庫欄位**（如 `carrid`，定義在 SE11）跟 **ABAP 程式裡的變數**（如 `gv_carrid`）。從 ABAP 這個「宿主語言」（Host Language）傳進 SQL 陳述式裡使用的變數，統稱**宿主變數（Host Variable）**——因為 Open SQL 本質上是內嵌（embedded）在 ABAP 裡的另一套語法，宿主變數就是從 ABAP 那邊「借」進來給 SQL 用的資料。

講義 6～25 一路用的舊式 Open SQL，並沒有強制標記宿主變數，編譯器靠**位置**猜：`WHERE carrid = gv_carrid` 裡，等號左邊照慣例是資料庫欄位、右邊是程式變數，位置關係單純，猜得準。但只靠位置猜，遇到下面兩種情況就會出問題：

- **欄位清單裡放算式、`CASE WHEN`、子查詢（Subquery）**：資料庫欄位跟程式變數可能混在同一個運算式裡（例如 `SUM( price * @lv_rate )`），編譯器無法只靠位置分辨誰是誰。
- **行內宣告 `@DATA(gt_x)`**：這個變數在陳述式當下才「憑空」誕生，編譯器更需要明確標記才知道「這是要宣告的宿主變數」，不是資料庫物件。

新式 Open SQL 的解法：宿主變數一律要加 `@` 前綴（`@gv_carrid`、`@p_carrid`、`@DATA(gt_x)`）。`@` 讓編譯器不用再猜——看到 `@` 就知道「這是 ABAP 那邊來的」，沒有 `@` 的識別字就是資料庫欄位，語意完全消歧義，也才撐得起算式、`CASE`、子查詢、行內宣告這些更複雜的語法。這是一次性的規則轉換：只要 `SELECT` 欄位清單用逗號分隔的新式寫法，句子裡**所有**宿主變數（含 `WHERE` / `INTO` 用到的）都要加 `@`，混用舊式 `INTO CORRESPONDING FIELDS OF TABLE itab`（不帶 `@`）也一樣要遵守——這是 `.claude/rules/sap-adt-mcp.md` 第 13 節記錄過的實測踩坑點。

```
SELECT scarr~carrid, scarr~carrname, sflight~connid, sflight~price,
       CASE WHEN sflight~price < 500  THEN 'LOW'
            WHEN sflight~price < 1500 THEN 'MID'
            ELSE 'HIGH'
```

```
      END AS price_level
  FROM scarr
  INNER JOIN sflight ON sflight~carrid = scarr~carrid
  ORDER BY scarr~carrid, sflight~connid
  INTO TABLE @DATA(gt_flight)
  UP TO 10 ROWS.
```

- `@DATA(gt_flight)`：不用先 `TYPES`/`DATA` 宣告內表，型別由 `SELECT` 清單自動推導——對照講義 11 的 JOIN 寫法，少了一段結構宣告；`@` 標記這是「要行內宣告的宿主變數」。
- `CASE WHEN ... THEN ... END AS alias`（講義 6 §3.7 有其他 SELECT 運算式）：在資料庫層就把分類算好，比撈回 ABAP 再用 `LOOP` + `IF` 判斷少一次資料搬移；`END AS` 給的別名會成為推導出結構的欄位名（`PRICE_LEVEL`）。
- 位置規則：新式寫法把 `INTO` 放到句尾時，`ORDER BY` 要寫在 `INTO` 之前，`UP TO n ROWS` 要接在 `INTO` 之後。傳統寫法則是 `INTO` 在 `WHERE` 之前、`ORDER BY` 在 `WHERE` 之後（講義 6 §3.5）。
- **`INTO` 放在整句最後，本身就是新式寫法的標誌**：傳統寫法的 `INTO` 只能放在「`SELECT` 欄位清單之後」或「`FROM` / `JOIN` 之後、`WHERE` 之前」；一旦像上面範例把 `INTO TABLE @DATA(...)` 放到 `ORDER BY` 後面，編譯器就進入新式模式，強制要求欄位清單用逗號分隔、宿主變數加 `@`（實測：傳統空格分隔的欄位清單搭配句尾 `INTO`，會報 `The elements in the "SELECT LIST" list must be separated using commas`）。所以講義 6～25 的傳統寫法，`INTO` 一律寫在 `FROM` 前面或緊接 `FROM` 之後。
- **聚合函數（Aggregate Function）裡可以直接放算式（呼應講義 20a 第 7 節）**：講義 20a 用傳統寫法時，`SUM( price * seatsocc )` 會報「must be separated using commas」，因為算式屬於新式語法；改成新式寫法就能在資料庫內直接算出講義 20 的各公司營收，不必撈明細再 `LOOP` 累加：

```
SELECT carrid,
       SUM( price * seatsocc ) AS revenue
  FROM sflight
  GROUP BY carrid
  ORDER BY carrid
  INTO TABLE @DATA(gt_rev).
```

3. COND：依條件回傳一個值（取代 IF/ELSEIF 賦值）

```
DATA(gv_level) = COND #( WHEN gv_score >= 90 THEN 'A'
                         WHEN gv_score >= 80 THEN 'B'
                         WHEN gv_score >= 60 THEN 'C'
                         ELSE 'D' ).
WRITE: / |成績等級：{ gv_level }|.
```

- 傳統寫法要先宣告 `gv_level`，再寫一串 `IF ... ELSEIF ... ELSE ... ENDIF`，`COND` 把「條件判斷」跟「賦值」合成一個運算式。
- `#` 代表型別由賦值目標（這裡是 `DATA(gv_level)` 新宣告的變數）自動推導，也可以寫明確型別如 `COND string( ... )`。
- 沒有任何 `WHEN` 命中、又沒寫 `ELSE`：**不會報錯**，結果是目標型別的初始值（空字串/0），容易誤判成「有算到但算錯」，要小心。`SWITCH` 沒寫 `ELSE` 也一樣回傳初始值；`CASE` 陳述式漏掉 `WHEN OTHERS` 則是比不中就什麼都不做，同樣不會報錯（2026-10-08 實測，`ZR_TR18_DATE_CHK`）。

4. SWITCH：依單一值做多分支對應（取代 CASE 陳述式賦值）

```
DATA gv_carrid TYPE s_carr_id VALUE 'LH'.

DATA(gv_carrname) = SWITCH #( gv_carrid
                        WHEN 'LH' THEN 'Lufthansa'
                        WHEN 'AA' THEN 'American Airlines'
                        WHEN 'UA' THEN 'United Airlines'
                        ELSE 'Unknown' ).
WRITE: / |航空公司：{ gv_carrname }|.
```

- `COND` 每個分支各自寫條件式（可以是不同欄位、不同運算子）；`SWITCH` 是針對**同一個值**比對多個候選值，對應傳統的 `CASE gv_carrid. WHEN 'LH'. ... ENDCASE.`。
- 選型原則：只是「這個值等於什麼就對應到什麼」→ `SWITCH`；判斷條件本身很複雜（範圍、多欄位、AND/OR）→ `COND`。

5. VALUE：建構結構 / 內表（取代逐筆 APPEND）

```
TYPES: BEGIN OF ty_item,
         matnr TYPE c LENGTH 10,
         qty   TYPE i,
       END OF ty_item,
       tt_item TYPE SORTED TABLE OF ty_item
                WITH UNIQUE KEY matnr
                WITH NON-UNIQUE SORTED KEY by_qty COMPONENTS qty.

DATA(gt_item) = VALUE tt_item( ( matnr = 'M001' qty = 10 )
                               ( matnr = 'M002' qty = 25 )
                               ( matnr = 'M003' qty = 5 ) ).
```

- 傳統寫法要宣告內表變數，再一筆一筆 `APPEND VALUE #( ... ) TO gt_item.`；`VALUE` 運算式可以整批用一句話建好初始內容，適合測試資料、固定清單。
- 這裡額外示範**次要索引（Secondary Key）**：`WITH NON-UNIQUE SORTED KEY by_qty COMPONENTS qty` 是給第 7 節 `FILTER` 用的，先在這裡跟主索引一起宣告。
- 用 `SORTED TABLE` 時，`VALUE` 給的資料順序要**符合主鍵（Primary Key）排序**（這裡 `matnr` 已經是 `M001 < M002 < M003` 遞增），不符合排序會執行期出錯。

6. REDUCE：迴圈累加成單一運算式（取代 LOOP + 累加變數）

```
DATA(gv_total_qty) = REDUCE i( INIT sum = 0
                               FOR wa IN gt_item
                               NEXT sum = sum + wa-qty ).
WRITE: / |總數量：{ gv_total_qty }|.
```

- 對照傳統寫法：`DATA gv_total_qty TYPE i. LOOP AT gt_item INTO DATA(wa). gv_total_qty = gv_total_qty + wa-qty. ENDLOOP.`——REDUCE 把「初始值 + 每輪怎麼更新」寫成一句。
- `INIT` 宣告累加變數的初始值（可以不只一個）；`FOR ... IN itab` 相當於 `LOOP AT`；`NEXT` 是每輪要執行的更新。
- 累加、串接字串、找最大值等「掃過整個內表濃縮成一個結果」的場景都適用；邏輯複雜（要中途 `EXIT`、多層判斷）時傳統 `LOOP` 可讀性通常更好，不要為了用而用。

7. FILTER：篩選子集合（取代 LOOP + WHERE 判斷 + APPEND）

```
DATA(gt_item_big) = FILTER #( gt_item USING KEY by_qty WHERE qty >= 10 ).

LOOP AT gt_item_big INTO DATA(gs_item_big).
  WRITE: / gs_item_big-matnr, gs_item_big-qty.
ENDLOOP.
```

- 對照傳統寫法：`LOOP AT gt_item INTO DATA(wa) WHERE qty >= 10. APPEND wa TO gt_item_big. ENDLOOP.`
- **關鍵限制**：`WHERE` 條件用到的欄位必須是**表格鍵值的一部分**（主鍵或用 `USING KEY` 指定的次要鍵），不能任意拿一個非鍵欄位做條件——這裡 `qty` 不是主鍵 `matnr`，所以第 5 節特地多宣告一個 `by_qty` 次要排序鍵給 `FILTER` 用。忘記這條規則、直接對非鍵欄位下 `FILTER` 會在啟用/編譯階段報錯，是這個運算子最容易踩的坑。

8. 舊寫法 vs 新寫法速查表

「詳見」欄是該語法點的「新式寫法對照」所在位置，有語法拆解與實測說明。

需求	傳統寫法	新式寫法	詳見
宣告並賦值	`DATA v TYPE i. v = 3.`	`DATA(v) = 3.`	講義 2 §1
依條件賦值	`IF ... ELSEIF ... ENDIF`	`COND #( WHEN ... THEN ... ELSE ... )`	講義 17 §2
依值對應賦值	`CASE v. WHEN ... ENDCASE.`	`SWITCH #( v WHEN ... THEN ... ELSE ... )`	講義 17 §3
字串拼接	`CONCATENATE a b INTO c`	`` c = | { a }{ b }| `` 或 `c = a && b`	講義 18 §1
取第 n 段	`SPLIT ... INTO v1 v2 v3`	`segment( val = s index = n sep = ',' )`	講義 18 §2
整理 / 取代 / 轉大寫	`CONDENSE`、`REPLACE`、`TRANSLATE`	`condense( )`、`replace( )`、`to_upper( )`	講義 18 §3

需求	傳統寫法	新式寫法	詳見
填結構	逐欄 `gs-f = ...`	`gs = VALUE #( f1 = ... f2 = ... )`	講義 3 §4
同名欄位對搬	`MOVE-CORRESPONDING a TO b`	`b = CORRESPONDING #( BASE ( b ) a )`	講義 3 §5.3
建內表	填 work area + `APPEND`	`VALUE tt( ( ... ) ( ... ) )`、`APPEND VALUE #( ... ) TO`	講義 4 §3
LOOP 的 work area	先 `DATA` 再 `LOOP ... INTO gs`	`LOOP ... INTO DATA(gs)`	講義 4 §4
讀單筆	`READ TABLE ... WITH KEY` / `INDEX`	`itab[ key = v ]`、`itab[ n ]`、`line_exists( )`	講義 4 §5
篩選子集合	`LOOP ... WHERE` + `APPEND`	`FILTER #( itab USING KEY k WHERE ... )`	講義 5 §3、本講第 7 節
迴圈累加	`LOOP` + 累加變數	`REDUCE type( INIT ... FOR ... NEXT ... )`	講義 5 §9
Field-Symbol	先 `FIELD-SYMBOLS` 再 `ASSIGNING`	`ASSIGNING FIELD-SYMBOL(<fs>)`、`ASSIGN itab[ ] TO FIELD-SYMBOL(<fs>)`	講義 16 §7
撈資料到內表	`DATA` 先宣告 + `SELECT ... INTO TABLE itab`	`SELECT ... INTO TABLE @DATA(itab)`	講義 6 §3.6
JOIN	先定義接收結構 + `INTO CORRESPONDING FIELDS OF TABLE`	`INTO TABLE @DATA(itab)`	講義 11 §2
群組小計（Control Break）	`AT NEW` / `AT END OF` / `SUM`	`LOOP AT ... GROUP BY` + `REDUCE`	講義 20 §5.1
聚合裡算營收	不支援，撈明細回來算	`SUM( price * seatsocc )`	講義 20a §7
寫入資料表	`INSERT ztab FROM gs`	`INSERT ztab FROM @gs`、`FROM @( VALUE #( ... ) )`	講義 21 §5

9. 常見錯誤與陷阱

症狀	原因
`SELECT` 新式逗號欄位清單編譯錯誤「must be escaped using @」	句中有宿主變數沒加 `@`——只要欄位清單用新式寫法，全句宿主變數都要跳脫，混用舊式 `INTO` 也一樣
`FILTER` 編譯期報錯，條件欄位無法使用	`WHERE` 條件欄位不是表格的主鍵或指定的 `USING KEY` 次要鍵
`COND`/`SWITCH` 沒有任何分支命中，結果不是預期的錯誤訊息而是空值/0	沒寫 `ELSE`，未命中時回傳初始值，不會報錯也不會 dump，容易誤判成邏輯算對了
字串模板 `{ }` 裡的方法呼叫編譯錯誤	只能放單一運算式 / `RETURNING` 方法呼叫，不能是有 `EXPORTING`/`IMPORTING` 多參數的方法
`VALUE` 建 `SORTED TABLE` 執行期 dump	給的資料順序不符合宣告的主鍵排序
`ORDER BY`/`UP TO n ROWS` 位置編譯錯誤	新式寫法 `INTO` 放句尾時：`ORDER BY` 在 `INTO` 之前，`UP TO n ROWS` 在 `INTO` 之後（傳統寫法的位置見講義 6 §3.5）

10. 課堂練習

完成 ex26：把講義 18 的字串範例、講義 11 的 JOIN 查詢、講義 17 的等第判斷、講義 20 的累加小計，各自改寫成新式寫法，並示範 `FILTER` 搭配次要鍵。